# ALU Logo Registry – Project Report

## Abstract

In this project I put the official ALU logo on the Ethereum blockchain so anyone can check whether a copy of the logo is the real one. I wrote two smart contracts: an ERC-721 contract that stores the logo's SHA-256 hash as an NFT, and an ERC-20 contract that splits ownership of the logo into 1,000,000 ALUT tokens. I tested both with eight automated tests and deployed them on a local blockchain and on the Sepolia testnet.

## Introduction

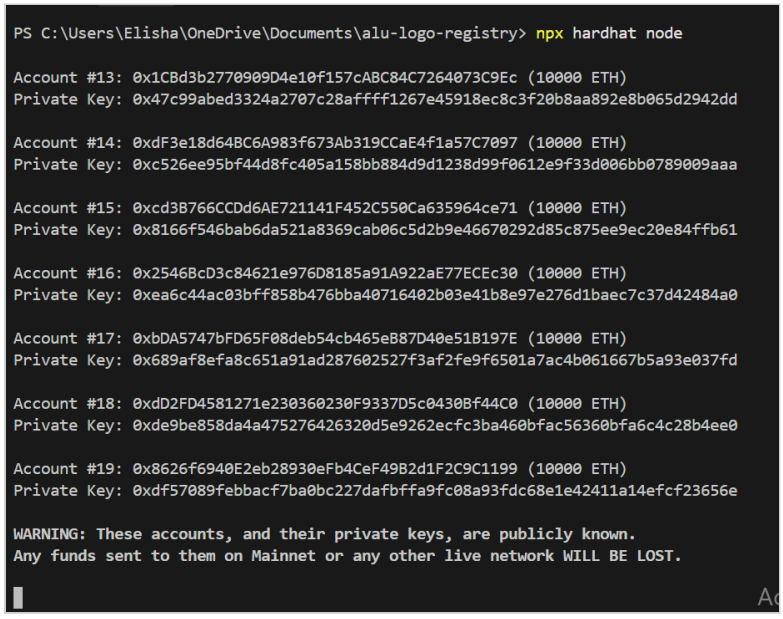
Anyone can download the ALU logo, change it a little and put it on a fake certificate or a scam post. People need an easy way to tell the real logo from a fake one.

A normal database could store the official logo, but whoever controls it can edit or delete records, so everyone else just has to trust them. Copyright registration protects the logo legally, but it's slow and you can't use it to check a file in a few seconds.

A blockchain solves both problems. Once the logo's hash is recorded, nobody can change it, not even ALU, and the record shows who registered it and when. Anyone can check their copy for free. ALU can also issue tokens showing who owns shares of the logo, which a database or a copyright office can't do.

## Blockchain Setup

I used Hardhat to run a local Ethereum blockchain on my laptop. I installed Node.js, created the project with npm, installed Hardhat, the Hardhat Toolbox and OpenZeppelin, and set the compiler to Solidity 0.8.24. Then I ran `npx hardhat node`, which starts the blockchain and prints 20 test wallets, each with 10,000 fake ETH.



A local blockchain is a private copy of Ethereum that only runs on my computer. Developers use one because transactions are instant and free, and you can restart it as often as you like. Testing on the real network would cost real money, and a buggy contract can't be deleted once it's there.

A wallet address is a 42-character code starting with `0x` that comes from a private key. On the blockchain the address is basically your account: it holds ETH and tokens, owns NFTs and shows up as the sender on every transaction you make.

## Smart Contract Explanation

### The logo hash

I downloaded the ALU logo from the ALU website and saved it as `alu-logo.png`. I wrote a small script, `scripts/hash-logo.js`, that reads the file and runs it through SHA-256 using Node's crypto module. I also checked it with PowerShell's `Get-FileHash` and got the same result. The file is named .png but it's actually a WebP image, so I registered it with the file type `image/webp`.

**ALU logo SHA-256:**
`0x102109d75b09daff117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5`

```
> alu-logo-registry@1.0.0 hash
> node scripts/hash-logo.js

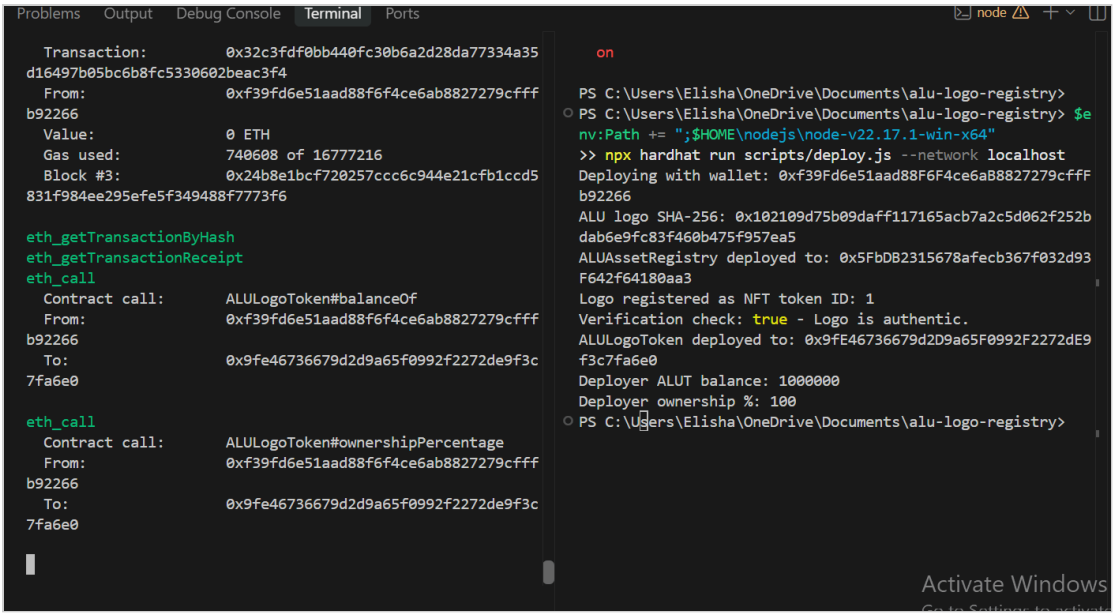
File:           C:\Users\Elisha\OneDrive\Documents\alu-logo-registry\alu-logo.png
SHA-256 (hex):  102109d75b09daff117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5
bytes32 value:  0x102109d75b09daff117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5
```

### ALUAssetRegistry

This contract extends OpenZeppelin's ERC-721. For each registered file it stores an `AssetMetadata` struct with the name, file type, SHA-256 hash, the wallet that registered it and the time. One mapping links each token ID to its details, and another one marks which hashes have already been used.

- **registerAsset()** takes a name, file type and hash. It rejects an empty hash or one that's already registered. Otherwise it increases the counter, saves the details, mints the NFT to the caller with `_safeMint()` and emits an `AssetRegistered` event. I save the data before minting, so the record is already correct if the receiving wallet runs any code during the mint.
- **verifyLogoIntegrity()** takes a token ID and a hash and compares it with the stored one. A match returns true and "Logo is authentic."; anything else returns false and "Warning: logo does not match." It's a view function, so it costs no gas.
- **getAsset()** returns all the stored details for a token.

The deploy script, `scripts/deploy.js`, hashes the logo, deploys the registry, registers the logo as token 1, checks it, and then deploys the token contract.




**What is a smart contract?** It's a program stored on the blockchain that runs exactly as written. A normal program runs on one company's server, and that company can change it or switch it off. A smart contract runs on every node, can't be changed once it's deployed, and anyone can see its data.

**What is an NFT, and why ERC-721?** An NFT is a token where each one is unique and has its own ID, and ERC-721 is the standard for them. It fits the logo because there's only one official ALU logo, so it should be one unique token with its own record.

**What is SHA-256?** It turns any file into a fixed 64-character fingerprint. Even a tiny change, like one pixel, gives a completely different result, because every bit of the file affects the whole calculation. So if two hashes match, the files are exactly the same.

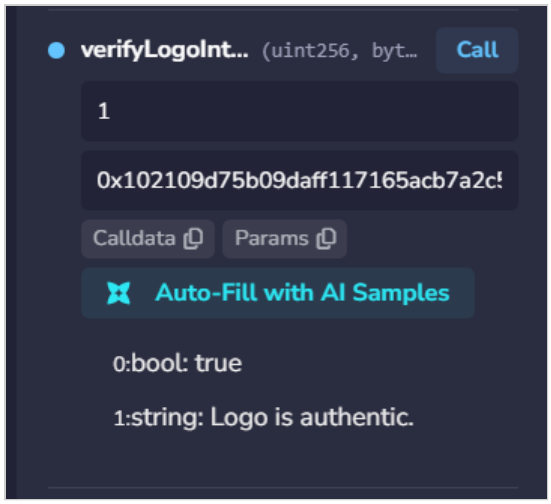
**Why can't the data be changed?** Every block includes the hash of the block before it. Changing an old record would change that block's hash and break every block after it, and the rest of the network would reject it. My contract also has no function to edit or delete a registration.

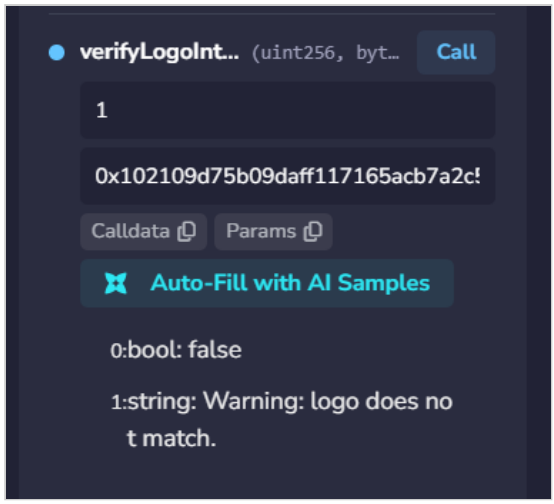
### Deployment on Sepolia

I also deployed both contracts to the Sepolia testnet using Remix IDE and MetaMask, with the compiler on 0.8.24 and the EVM version set to cancun.

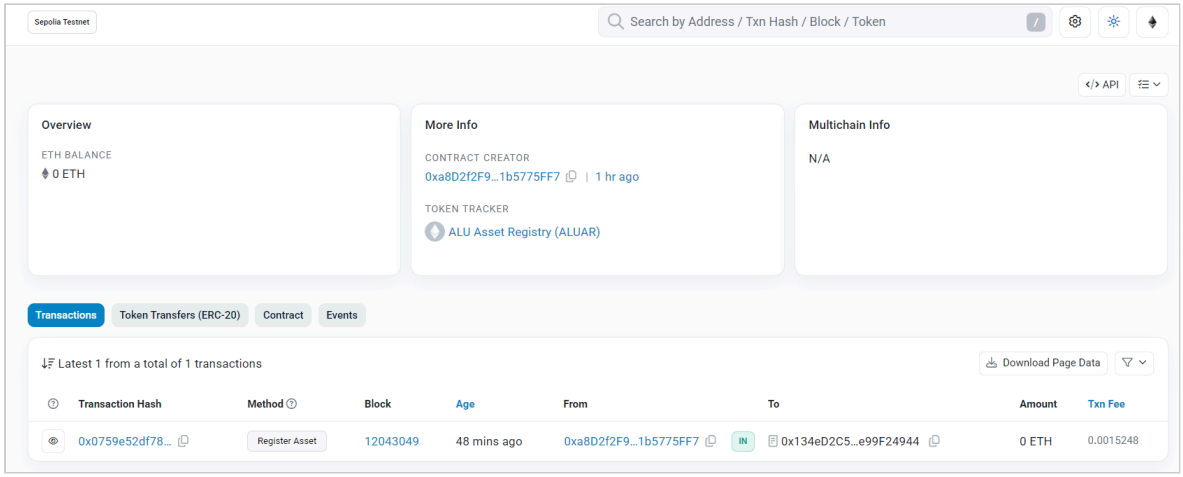
| Contract | Address |
|---|---|
| ALUAssetRegistry | 0x134eD2C5d179CcE082fE0A3823BDC63e99F24944 |
| ALULogoToken | 0x852207366C6e07F8bDad37817957F80e2Ca88e60 |
| My wallet | 0xa8d2f2f9c46021c7dac6e7ba450a39e1b5775ff7 |

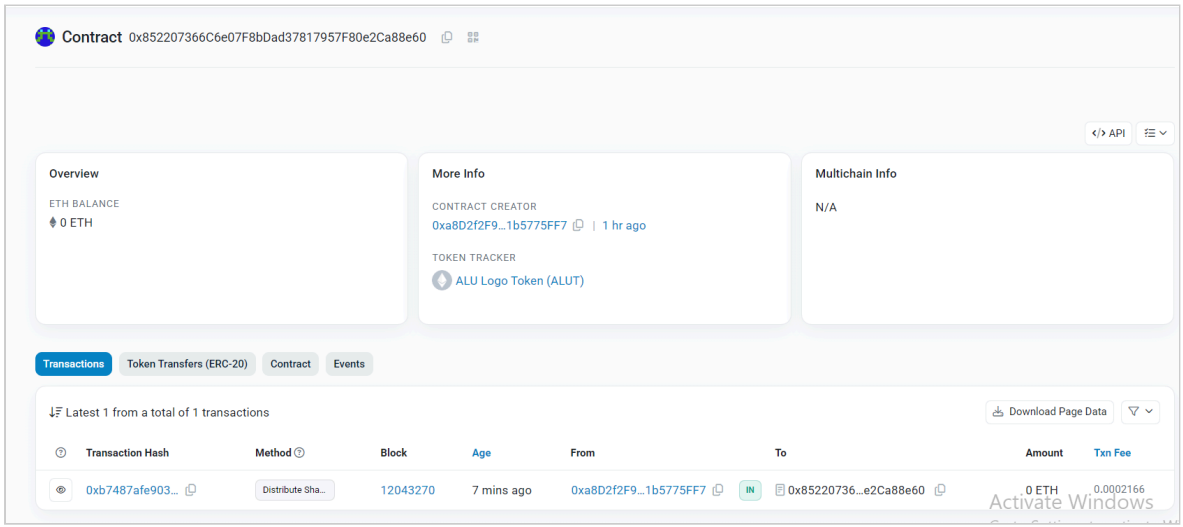
After registering the logo, verifyLogoIntegrity returned true with the real hash and false with a slightly changed one:




On Etherscan, both contracts show my wallet as the creator, along with the Register Asset and Distribute Shares transactions:



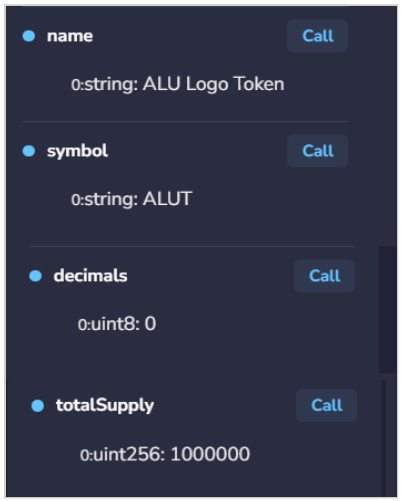

## Tokenisation

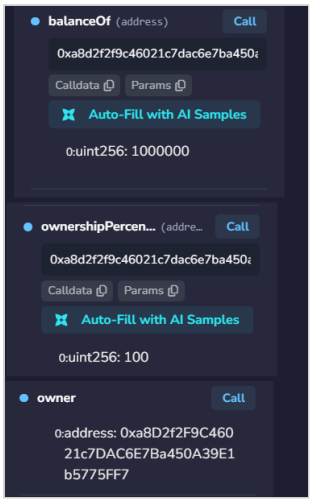
Tokenisation means turning ownership of something into tokens on a blockchain. Here the ALU logo is split into 1,000,000 ALUT tokens, and each token is one share.

`ALULogoToken` extends ERC-20 and Ownable. It's called "ALU Logo Token" with the symbol ALUT. When it's deployed, all 1,000,000 tokens go to the logo owner, and there's no way to mint more. I set decimals to 0 so one token is one whole share.

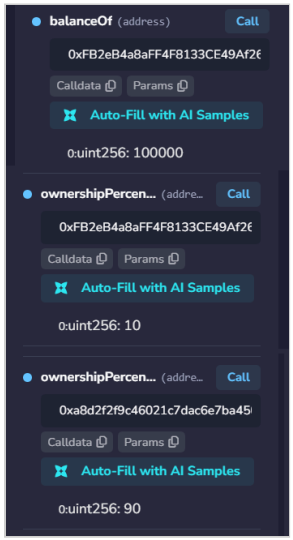
- **distributeShares()** can only be called by the owner. It checks the amount is more than zero and sends that many ALUT to the recipient.
- **ownershipPercentage()** returns a wallet's share as a whole number, so 500,000 tokens returns 50.

On Sepolia the token showed the right name, symbol and supply, and my wallet started with all 1,000,000 ALUT:




Then I sent 100,000 ALUT to a second account playing the role of a faculty member. It now owns 10% and I own 90%:




**Who at ALU would hold ALUT?** I'd keep most of it with ALU administration, around 60%, so they stay in control of the brand. Faculty could get about 20%, students and alumni 15%, and the marketing team 5%, since they all help build and represent the ALU name.

**ERC-721 vs ERC-20.** ERC-721 tokens are each unique, while ERC-20 tokens are all the same, like coins. I used ERC-721 for the logo because there's only one official logo. I used ERC-20 for the shares because one share is the same as any other, and people need to hold different amounts.

**What does 100,000 ALUT mean?** A faculty member with 100,000 ALUT owns 10% of the logo. If ALU earned money from licensing it, they could get 10% of that, or have 10% of the vote on how it's used. The blockchain only records the 10%, so ALU would have to agree separately on what rights come with it.

## Test Results

All eight tests pass with `npx hardhat test`. Tests 1–5 cover the registry and tests 6–8 cover the token.

```
ALU Logo Protection
  ALUAssetRegistry
    ✓ 1. registers the ALU logo and returns a token ID (1571ms)
    ✓ 2. rejects registering the same hash a second time (73ms)
    ✓ 3. verifyLogoIntegrity returns true for the correct hash
    ✓ 4. verifyLogoIntegrity returns false for a wrong hash
    ✓ 5. getAsset returns the right name and file type (40ms)
  ALULogoToken
    ✓ 6. mints all 1,000,000 ALUT to the logo owner (67ms)
    ✓ 7. distributeShares sends tokens to a recipient
    ✓ 8. ownershipPercentage returns the right percentage


8 passing (2s)
```

## Conclusion

This project showed me how a hash and a blockchain together can prove a file is genuine without storing the file itself. I learned how to set up Hardhat, write contracts with OpenZeppelin, test them, and deploy them to a real testnet with Remix and MetaMask. I also learned that small details matter, like setting the EVM version to cancun to fix a compile error.

If I had more time, I'd verify the source code on Etherscan, store the logo on IPFS so wallets can show it, link the token contract to the NFT, and build a web page where anyone can upload a logo and check it.

## GitHub and Video

- GitHub: https://github.com/relisha1/alu-logo-registry
- Video: [ADD YOUR VIDEO LINK]